UNIVERSITY OF CANBERRA
AUSTRALIA'S CAPITAL UNIVERSITY

Faculty of Science and Technology

Student ID numbers	U3317210, u3287346
Unit name	Software Technology 1
Unit number	4483
Name of lecturer/tutor	Unit Lecturer: Girija Chetty Unit Tutor: Pranav Gupta
Assignment name	Assignment 3: Part A – Implementation and Summary
Due date	17/05/2026

Student declaration

We certify that the attached assignment is our own work. Material drawn from other sources has been appropriately and fully acknowledged as to author/creator, source and other bibliographic details. Such referencing may need to meet unit-specific requirements as to format and style.

Date of submission: **16/05/2026**

Assignment 3 - Macroinvertebrate Image Analysis Project

Project goal

The project goals are to analyse stream macroinvertebrate image dataset from Kaggle and organise the images into a structured table that performs exploratory data analysis (EDA) to explore how many images exist per class and what sizes they are. Then the goal is to process the images by converting every single image, whether it is coloured or greyscale into the same format 128x128 pixels before being trained to the model. Then the images will be used to create 6 different EDA including class distribution, image size distribution, box plot, brightness by class, average RGB channels across the dataset, and sample grid of the image dataset. The last goal is to build a tkinter app that allows a user to select an image and predict its class with a confidence percentage.

Summary of the system design

The Project is designed as a modular Python application where each class has one clear responsibility. The work is divided across separate classes and files that work together as a complete system. There are two main stages. Stage 1 handles the exploratory data analysis and stage 3 provides the desktop graphical user interface that displays the results.

For stage 1, the dataset is scanned by the datasetIndexer class which takes data from data/raw/stream_macroinvertebrates/ folder and builds a structured pandas DataFrame containing the file path, width, label, height and the number of images per class. The dataFrame is then passed to the EDAService class which generates four key visual outputs that includes the class distribution bar chart, image size distribution histogram, a sample image grid, and a box plot that shows image dimensions per class. All outputs are saved to the outputs/eda/ folder as PNG files.

The MacroApp class provides a Tkinter desktop interface that loads and displays the EDA outputs generated in stage 1. The UI consists of two sections. The left sidebar contains navigation buttons for switching between the four EDA charts, a dataset summary which shows total images, total classes, mean and width height. On the right side of the UI, the selected chart displays with a title that updates automatically. Lastly, there is a export button that allows the user to save the chart to a location of their choice.

The application also handles error like if a chart file is not found in the outputs/eda/ folder, then the UI displays a clear error message to the user.

Class and module overview

Class	Responsibility
MacroApp	MacroApp class is used in stage 3 to provide a desktop graphical interface for viewing the EDA outputs generated in stage 1. Tkinter is used to make the class the main application window. When app.py is launched through “python -m src.main”, the tkinter application loads which then allows the user to interact through the buttons and navigation side bar to display the three EDA charts that was saved in the folders outputs/eda/.

Dataset Indexer	Image Record is a data class defined in records.py that is responsible for storing the core metadata of a single macroinvertebrate image. It represents each image as a structured Python object containing five fields: file path, label, width, height, and channels. Each image in the dataset is wrapped into an Image Record inside build_dataframe() before being added to the pandas Data Frame.
EDA Service	EDAService is the core class defined in eda_service.py that is responsible for all data scanning, analysis, and chart generation. It takes the dataset path and output folder as inputs. The build_dataframe method scans the dataset and builds a pandas DataFrame using OpenCV and NumPy. Six chart-generating methods produce all EDA visualisations which are saved to the outputs/eda folder. The run_all method orchestrates the entire pipeline in sequence.

Python packages and libraries used and why

Pandas： Pandas was used to store all image metadata in a structured DataFrame. After the build_dataframe method processes each image, all the information including file path, label, width, height, channels, and brightness is collected into a pandas DataFrame stored as self.df. This DataFrame is then shared across all six chart-generating methods, so every method reads from the same data source. The value_counts, groupby, median, and sort_values functions from Pandas are also used inside the chart methods to sort and group data before plotting.

Numpy ：NumPy was used for all pixel-level numerical computation. In the build_dataframe method, the mean function from NumPy computes the mean pixel brightness across all channels for each image. In the save_rgb_channels method, NumPy is used again to compute the average pixel value for each of the three colour channels, Red, Green, and Blue separately across the entire dataset.

OpenCv python： OpenCV was used in three different methods. In build_dataframe, the imread function from OpenCV reads each image file from the dataset and returns a NumPy array. From this array, we extract the width, height, and number of channels using the shape attribute. In save_rgb_channels, OpenCV re-reads each image to separate and calculate the average value of each color channel individually. In save_sample_grid, the cvtColor function from OpenCV converts images from BGR to RGB before displaying them, since OpenCV loads images in BGR format by default.

Matplotlib： Matplotlib was used as the foundation for generating all six static chart outputs. The save_size_plots method uses the subplots function from Matplotlib to create two side-by-side histograms. The save_boxplot method uses the figure function to create a box plot comparing image dimensions. The save_rgb_channels method uses the bar function to plot the average RGB channel values with colour-coded bars. The save_sample_grid method uses a 3 by 3 grid of subplots to arrange nine sample images using the imshow function. All charts are saved using the savefig function and closed with the close function to free memory.

Seaborn ： Seaborn was used on top of Matplotlib to generate the more complex statistical charts. The save_class_distribution method uses the countplot function from Seaborn to create a bar chart of images per class, sorted by count. The save_size_plots method uses the histplot function to generate the width and height distribution histograms. The save_boxplot method uses the boxplot function to compare the spread of image dimensions. The save_brightness_by_class method uses the boxplot function again to show brightness distribution grouped by species class, sorted by median brightness.

Plotly： Plotly was used to generate one additional interactive HTML chart. In the save_plotly_chart method, px.histogram creates an interactive bar chart of class counts. Unlike the static Matplotlib charts, this output is saved as plotly_classes.html using fig.write_html, and can be opened in a browser with hover tooltips and zoom functionality.

tkinter

Tkinter was used to build the desktop GUI for stage 3. This was used to create the entire visual structure of the application including the main window, header banner, sidebar navigation panel, the chart display area and all buttons and labels. Components like tk.Frame was used to create the container sections and divide the layout, tk.Label to display texts and images, and tk.Button to create clickable buttons for the user to open the EDA charts. Our group decided to complete stage 3 using tkinter instead of menu-driven console structure because we were more familiar with tkinter as it was used more during lectures, tutorial tasks and quizzes.

Pillow

Pillow was used to open the EDA chart images and display them inside the tkinter window. It was chosen because tkinter cannot open or display PNG images files which is why pillow was used to load the EDA charts from the folder outputs/eda/ and convert them into a format that tkinter could display inside the application.

Pathlib

This package was used to handle file paths for the EDA charts. It was chosen because it provides a cleaner and more readable way of working with file paths.

Filedialog and messagebox

Both packages are from Python's built in tkinter library which was used to open a save dialog window for the chart export feature and to display popup warnings or confirmation window to the user.

Key features implemented

stage 1 key features

Dataset Indexing: Scans the macroinvertebrate image dataset recursively using os.listdir(), extracts metadata (file path, class label, width, height, channels, brightness) for each image using OpenCV, and stores all records in a structured Pandas DataFrame.

Class Distribution Analysis: Generates a bar chart showing the number of images per class, revealing class imbalance across the dataset to inform model training decisions in Stage 2.

Image Size Analysis: Produces width and height histograms and a box plot to identify inconsistent image dimensions across the dataset.

RGB Channel Analysis: Computes mean pixel values for each colour channel (Red, Green, Blue) across all images using NumPy, and visualises the result as a colour-coded bar chart.

Brightness Analysis: Calculates mean brightness per image and displays per-class brightness distributions as a box plot, sorted by median brightness to highlight lighting variation between classes.

Sample Image Grid: Randomly samples 9 images (random_state=42 for reproducibility) and displays them in a 3×3 grid with class labels, providing a quick visual inspection of the dataset.

Interactive Visualisation: Saves an interactive Plotly HTML chart of class distribution, allowing dynamic exploration of the data in a web browser.

stage 3 key features

Desktop GUI application: The application opens automatically and display the EDA outputs in an interactive window.

Sidebar navigation panel: A sidebar navigation panel was implemented on the left of the window with three buttons for the user to interact with which includes the class distribution, image sizes, and sample grid charts. When a button is clicked, the colour of the button changes colour to pink so the user can know which chart they have selected. The main purpose of this feature is to provide clear visual feedback to the user.

EDA chart: Once a button a clicked, the application displays that EDA chart. It allows the user to switch between the three charts by clicking the sidebar buttons which the top left of the chart, the it displays the name of the selected chart.

Export button: Users can export any chart on their desktop by allowing the user to save the current display chart to a location of their choice. That chart is then saved as a PNG file.

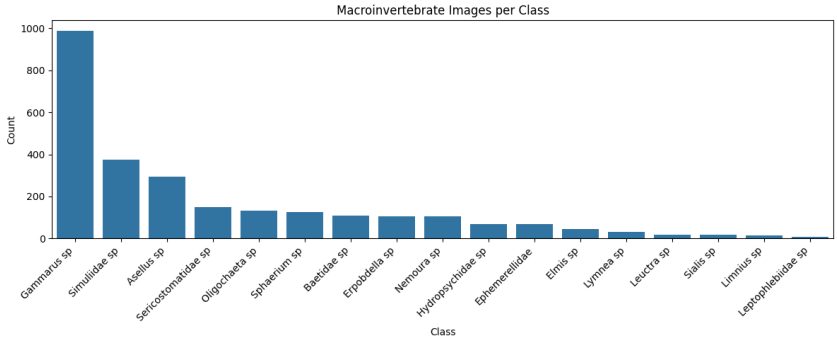
Dataset summary: User can also see information about dataset stats that includes total images, total classes, mean width and height of stream Macroinvertebrate. This is displayed at the bottom of the navigation panel so it can give the user a quick overview of the dataset.

Error handling: If one of the charts selected is not found in the outputs/eda/ then a error message appears to notify users to run stage 1 first.

Screenshots or example outputs

Stage 1

```
===== Running Stage 1 EDA =====
DEBUG: DataFrame shape = (2665, 6)
DEBUG: Columns = ['path', 'label', 'width', 'height', 'channels', 'brightness']
===== STAGE 1 EDA SUMMARY =====
  total_images: 2665
  total_classes: 17
  mean_width: 599.2660412757974
  mean_height: 389.98198874296435
  mean_brightness: 181.29920450281426
```



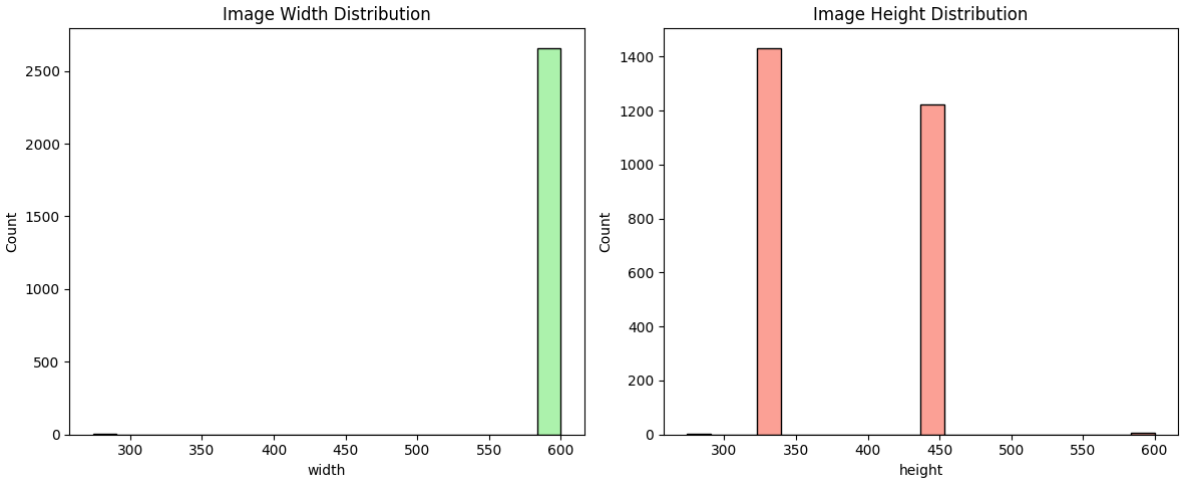
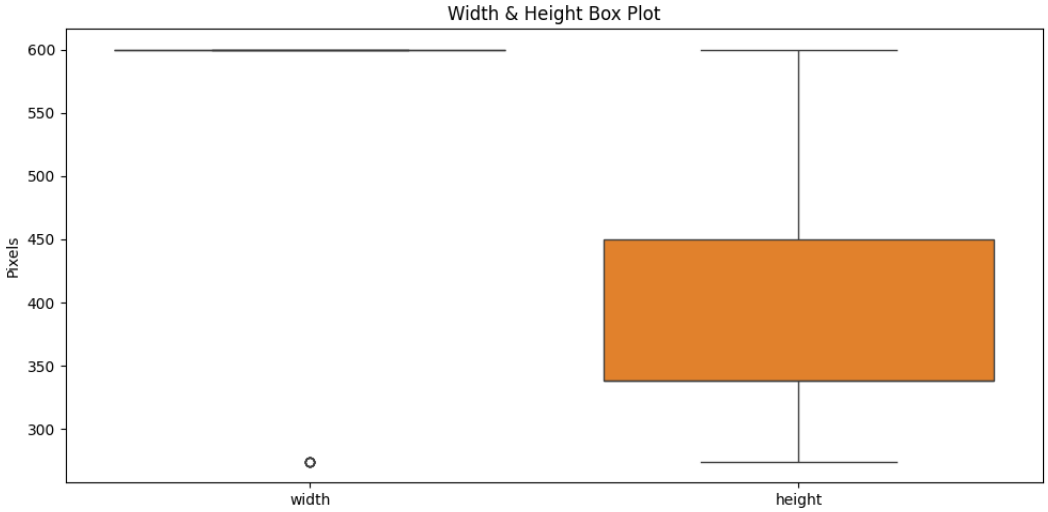
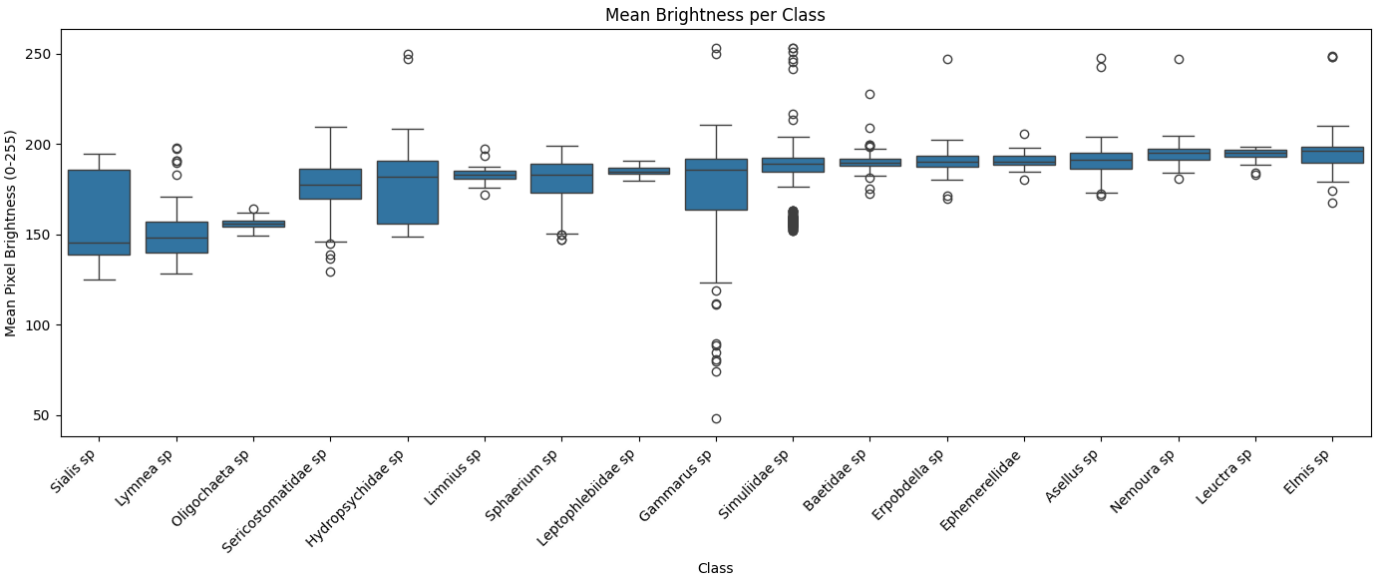
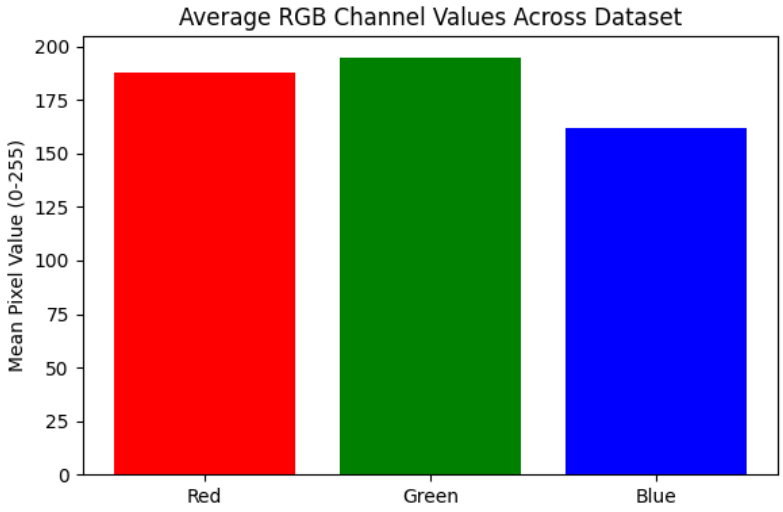


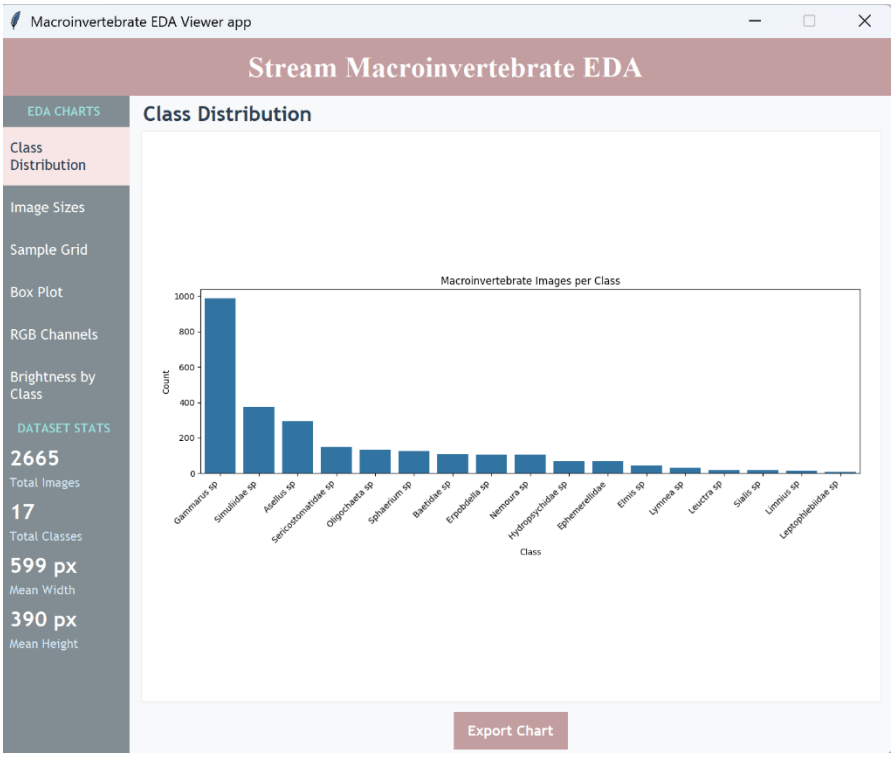
Image Width Distribution
Count
width
Image Height Distribution
Count
height


Width & Height Box Plot
Pixels
width
height


Mean Brightness per Class
Mean Pixel Brightness (0-255)
Sialis sp
Lymnea sp
Oligochaeta sp
Sericostomatidae sp
Hydropsychidae sp
Limnius sp
Sphaerium sp
Leptophlebiidae sp
Gammarus sp
Simuliidae sp
Baetidae sp
Erpobdella sp
Ephemerellidae
Asellus sp
Nemoura sp
Leuctra sp
Elmis sp
Class



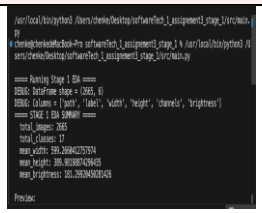
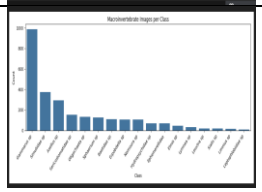
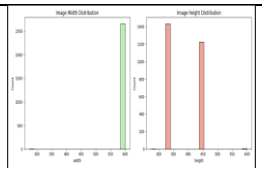
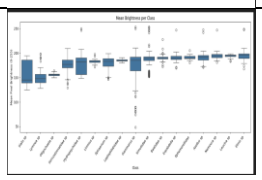
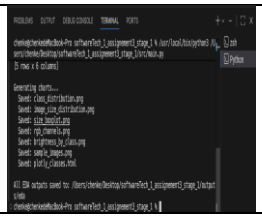
Stage 3



By default, class distribution appears first when the app is launched. On the left navigation panel, users can select any of the six buttons to display the EDA.

Testing summary

stage 1

Scenario	Input	Expected result	Actual result	Evidence
Dataset loads successfully and DataFrame is built	DATASET_PATH pointing to stream_macroinvertebrates/ with labelled subfolders	build_dataframe() returns a DataFrame with 6 columns (path, label, width, height, channels, brightness) and at least one row per class	Pass	
Dataset summary statistics are printed correctly	Fully loaded DataFrame	print_summary() prints all 5 statistics: total_images, total_classes, mean_width, mean_height, mean_brightness	Pass	
Class distribution chart is generated and saved	DataFrame with multiple class labels	class_distribution.png saved to outputs/eda/. Chart displays one bar per class with counts on Y-axis	Pass	
Image size distribution histograms are saved	DataFrame width and height columns	image_size_distribution.png saved with two subplots: Image Width Distribution and Image Height Distribution	Pass	
Sample image grid is saved with correct colours	9 random images sampled from DataFrame	sample_images.png saved. 3x3 grid showing images with natural colours (BGR converted to RGB) and class label titles	Pass	
Interactive Plotly chart opens and responds to hover	DataFrame label column written to outputs/eda/plotly_classes.html	HTML file opens in browser and displays an interactive class distribution chart with Plotly toolbar	Pass	
Non-image files in dataset folder are ignored	A notes.txt file placed inside one class subfolder alongside valid images	notes.txt is skipped by the extension check. DataFrame row count equals valid image count only. No error is raised	Pass	

stage 3

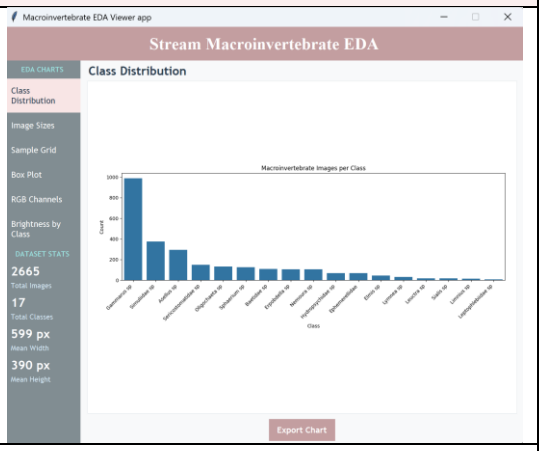
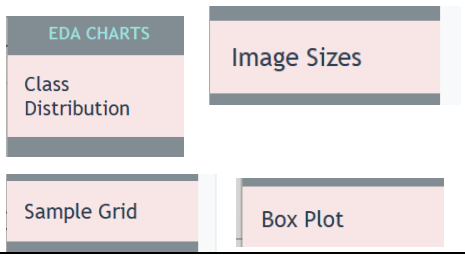
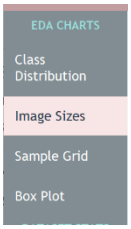
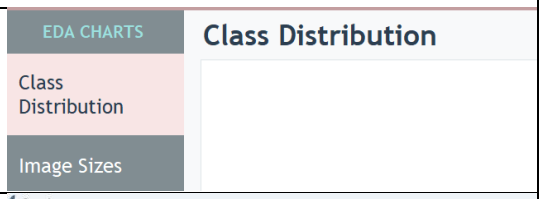
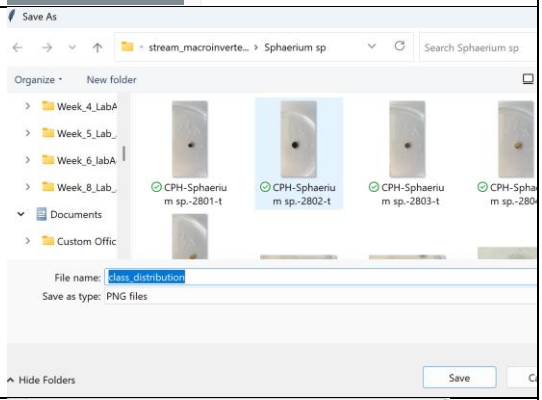
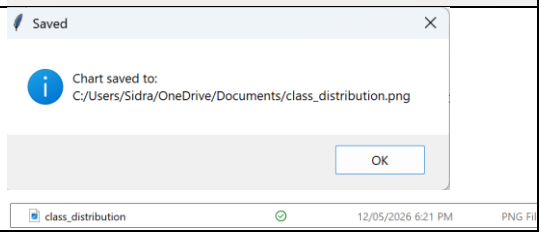
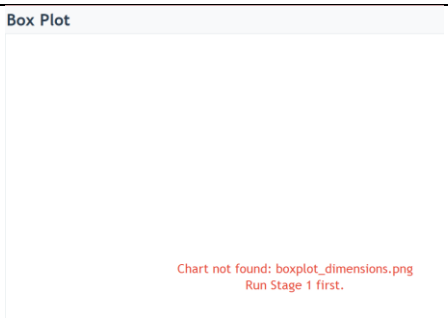
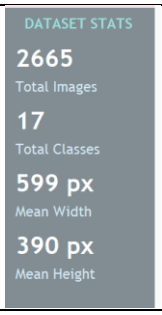
Scenario	Input	Expected result	Actual result	Evidence
App launches successfully	run python -m src.main	GUI window opens with class distribution chart loaded by default	Pass	
All buttons change colour	Click class distribution, image sizes, sample grid, and box plot buttons.	All 6 buttons should change to pink	Pass	
Button highlight resets once another button is clicked.	First click one button then another.	The previous button selected returns to grey and new button turns pink.	Pass	
Chart title updates	Click different sidebar buttons	Title above chart updates to match the selected chart name	Pass	
Export chart	Select a chart then click export button	File window should open and chart saves correctly as a PNG file	Pass	
Export to a location	Export chart to desktop then in documents	Chart saves correctly to both locations and the popup window s	pass	

Chart not found error	Delete box plot chart from outout/eda/ then click box plot button in the side bar of the app.	Error message displays that says chart not found, run stage 1 first.	Pass	
Dataset summary is visible on the sidebar.	Open the application.	Sidebar shows correct stats which is 2665 images, 17 classes, 599px width, and 390px height.	Pass	

Acknowledgement and explanation of code

Stage 1

The code for main.py was reused and adapted from the guidance document (view figure 1 and 2). The purpose of main.py is to run workflowService pipeline to generate the EDA charts and then open the MacropApp.

Then your `main.py` stays small and consistent with the same project structure:

```python
from src.services.workflow_service import WorkflowService

def main() -> None:
    """Run the default non-interactive project workflow."""

    workflow = WorkflowService()
    workflow.run_full_pipeline()

if __name__ == "__main__":
    main()
```

Figure 1
screenshot from: full guidance and coding examples.pdf

```python
import sys, os
sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
from src.config import DATASET_PATH, OUTPUT_FOLDER
from src.services.eda_service import EDAService
from src.app import MacroApp


def main():
    eda = EDAService(DATASET_PATH, OUTPUT_FOLDER)
    eda.run_all()

    # Launch Stage 3 GUI
    app = MacroApp()
    app.mainloop()

if __name__ == "__main__":
    main()
```

Figure 2: actual main.py code

stage 3

Initially, the code from step 9: build stage 3 deployment using tkinter was used to begin with stage 3 which was later changed to incorporate other functions.

The first method runs automatically when the app is created. It sets up the window size, title, and background. It also defines which charts to display in the navigation panel (view figure 3).

```python
def __init__(self) -> None:
    super().__init__()
    self.title("Macroinvertebrate EDA Viewer app")
    self.geometry("1500x1200")
    self.configure(bg=self.WINDOW_BG)
    self.resizable(False, False)

    self.chart_options = {
        "Class Distribution": "class_distribution.png",
        "Image Size Distribution": "image_size_distribution.png",
        "Sample Image Grid": "sample_grid.png",
    }
    self.chart_keys = list(self.chart_options.keys())
    self.current_index = 0

    self.create_a
```

Figure 3

The second method create_app builds the overall layout such as the header at the top and then places the sidebar and chart area (view figure 4).

```python
def create_app (self) -> None:
    self.create_header()

    content = tk.Frame(self, bg=self.WINDOW_BG)
    content.pack(fill="both", expand=True)

    self.create_sidebar(content)
    self.chart_display(content)
```

Figure 4

The third method is create_header which builds the pink header at the top. Inside the frame, label widgets is placed for the title, font type and size, and colour of the font (view figure 5).

```python
def create_header(self) -> None:
    header = tk.Frame(self, bg=self.HEADER_PINK, pady=18)
    header.pack(fill="x")

    tk.Label(
        header,
        text="Stream Macroinvertebrate EDA",
        font=("Times New Roman", 18, "bold"),
        bg=self.HEADER_PINK,
        fg="white"
    ).pack()
```

Figure 5

The create_sidebar() method creates the left navigation panel. The chart buttons includes six buttons, one for each EDA chart.

```python
def create_sidebar(self, parent) -> None:
    sidebar = tk.Frame(parent, bg=self.SIDEBAR_BG, width=220)
    sidebar.pack(side="left", fill="y")
    sidebar.pack_propagate(False)

    # EDA chart title at the top of the side bar

    tk.Label(
        sidebar,
        text="EDA CHARTS",
        font=("Trebuchet MS", 8, "bold"),
        bg=self.SIDEBAR_BG,
        fg="#9ce1df",
        pady=10
    ).pack()
    self.chart_buttons = {}
```

```python
charts = [
    ("Class Distribution", "Class Distribution"),
    ("Image Sizes", "Image Size Distribution"),
    ("Sample Grid", "Sample Image Grid"),
    ("Box Plot", "Box Plot"),
    ("RGB Channels", "RGB Channels"),
    ("Brightness by Class", "Brightness by Class"),
]
```

Figure 6

The select_chart() method handles the button highlighting and chart display when user clicks a one of the chart buttons. It also changes the text to dark once its highlighted (view figure 7)

```python
def select_chart(self, key: str) -> None:
    """Highlight the selected button and display the chart."""

    # reset all buttons to default colour
    for k, btn in self.chart_buttons.items():
        btn.configure(bg=self.SIDEBAR_BG, fg="white")

    # button colour changes to pink when selected
    self.chart_buttons[key].configure(
        bg=self.SELECTED_PINK,
        fg=self.TEXT_DARK)

    # display the chart
    self.change_chart(key)
```

Figure 7

export feature

The export_chart method in app.py was adapted from a code example found online that used asksaveasfilename from tkinter to save a file. As shown in figure 8, the original code used asksaveasfilename to save text content, writing it to a file in text mode. This was adapted in app.py to instead copy an existing PNG chart file, reading it in binary mode “rb” from the EDA output and writing it to the location the user selects in “wb” (view figure 9).

```
import tkinter as tk
import tkinter.filedialog as filedialog

def save_file():
   file_path = filedialog.asksaveasfilename(defaultextension=".txt", filetypes=[("Text
Files", "*.txt"), ("All Files", "*.*")])
   if file_path:
      text_content = text_widget.get("1.0", tk.END)
      with open(file_path, "w") as file:
         file.write(text_content)

window = tk.Tk()
window.title("Text Editor")

text_widget = tk.Text(window)
text_widget.pack()

save_button = tk.Button(window, text="Save", command=save_file)
save_button.pack()

window.mainloop()
```

Figure 8:(Kumar, n.d.)

```
def export_chart(self) -> None:
    """Save the current chart to a location chosen by the user."""
    key = self.chart_keys[self.current_index]
    filename = self.chart_options[key]
    chart_path = EDA_OUTPUT_DIR / filename

    if not chart_path.exists():
        messagebox.showwarning("Not Found", "Chart file not found.")
        return
    save_path = filedialog.asksaveasfilename(
        defaultextension=".png",
        filetypes=[("PNG files", "*.png")],
        initialfile=filename)

    if not save_path:
        return

# read the chart file and write it to the new location
    with open(chart_path, "rb") as file:
        data = file.read()
    with open(save_path, "wb") as file:
        file.write(data)

    messagebox.showinfo("Saved", f"Chart saved to:\n{save_path}")
```

Figure 9: app.py

Summary of how work was divided among group members

Member	Main responsibility
Lily	Stage 1: EDA
Sidra	Stage 3: tkinter GUI

Collaboration and Integration

Both of us agreed on a project folder structure from the Assignment 3 guidance document provided to us. This was an important first step because it meant both of us knew exactly where files would be located and what they would be called (view figure 10)

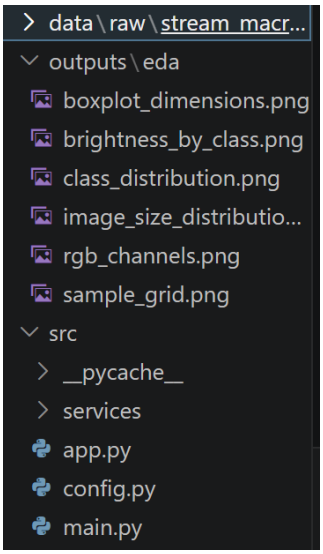


Figure 10: folder structure

We used GitHub as our primary tool for collaboration throughout the project. Each of us worked on our tasks and then uploaded our code to a shared GitHub repository which allowed us to get access, review, and build on each other's work at any time. Lili completed stage 1 EDA and then I used her code to build stage 3. I integrated stage 1 by using EDA_OUTPUT_DIR from config.py in app.py so that the UI application knew where to find the charts. Then the output charts filenames were used to add it in the chart_options dictionary in app.py (view figure 11).

Finally, in main.py we connected both stages where stage 1 must always run first before the app opens so that the chart files exist when the app tries to load them.

```
# EDA chart options from stage 1
    self.chart_options = {
        "Class Distribution": "class_distribution.png",
        "Image Size Distribution": "image_size_distribution.png",
        "Sample Image Grid": "sample_grid.png",
        "Box Plot": "boxplot_dimensions.png",
        "RGB Channels": "rgb_channels.png",
        "Brightness
    }
```

Figure 11: code from app.py

We also did regular communication throughout the project to discuss design decisions, resolve errors, and divide tasks. This was done through group chats and group meetings.

References

Bradski, G. (2000). The OpenCV library. *Dr. Dobb's Journal of Software Tools*. https://github.com/opencv/opencv

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. Computing in Science & Engineering, 9(3), 90–95. https://doi.org/10.1109/MCSE.2007.55

Kumar, B. (n.d.). Python Tkinter Save Text To File - Python Guides. [online] Available at: https://pythonguides.com/python-tkinter-save-text-to-file/. [accessed on 9 May 2026]